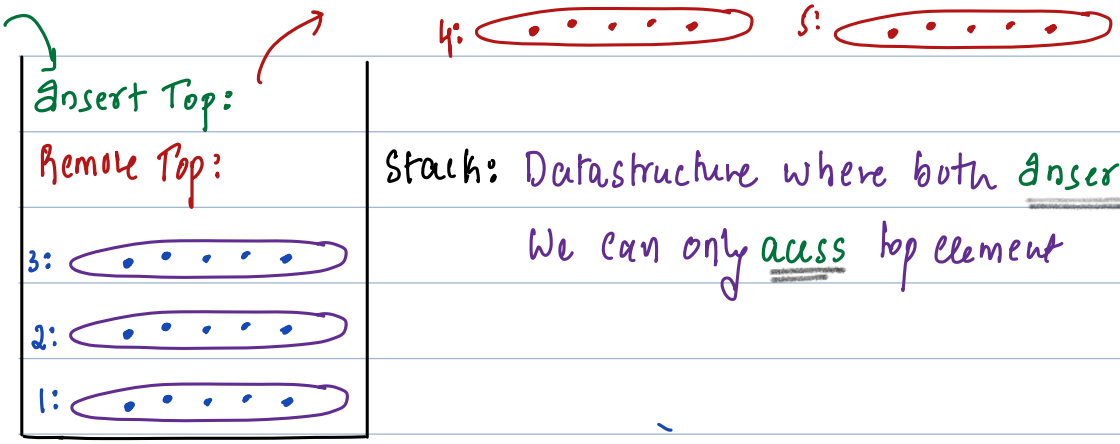
Todays Content

1. Introduction to stack }
2. Reference variable {
3. Types of memory }
4. Examples & Quizes. {

Idli Cooker:



Stack: Datastructure where both Insert & Delete from top.
We can only access top element

Function Calls:

1. When we call a function, it is inserted at top of stack
2. When function call execution is completed, it will be removed from stack.
   a. Execute Return statement or
   b. Execute last line of function.

Ex1:

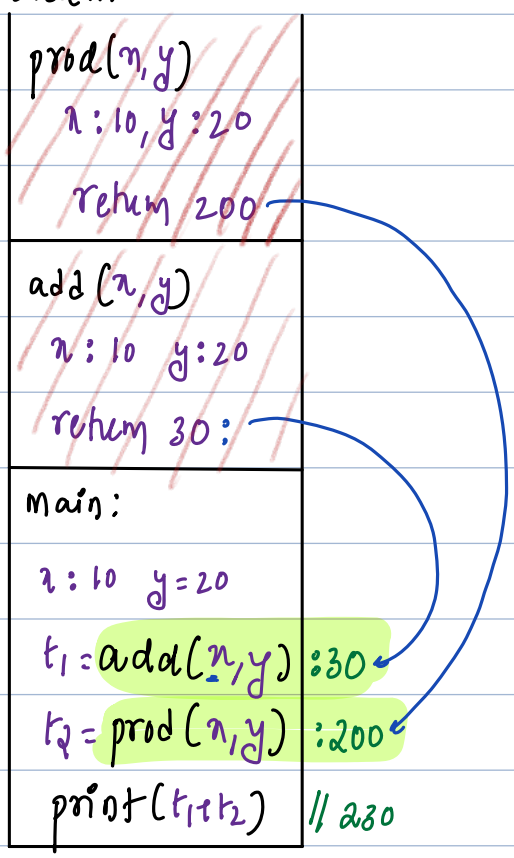
```
static int product(int x, int y){
    return x*y;
}

static int add(int x, int y){
    return x+y;
}

public static void main(){
    int x=10, y=20;
    int t1= add(x,y); 30
    int t2= prod(x,y);
    print(t1+t2) //230
}
```

OS:

Stack:

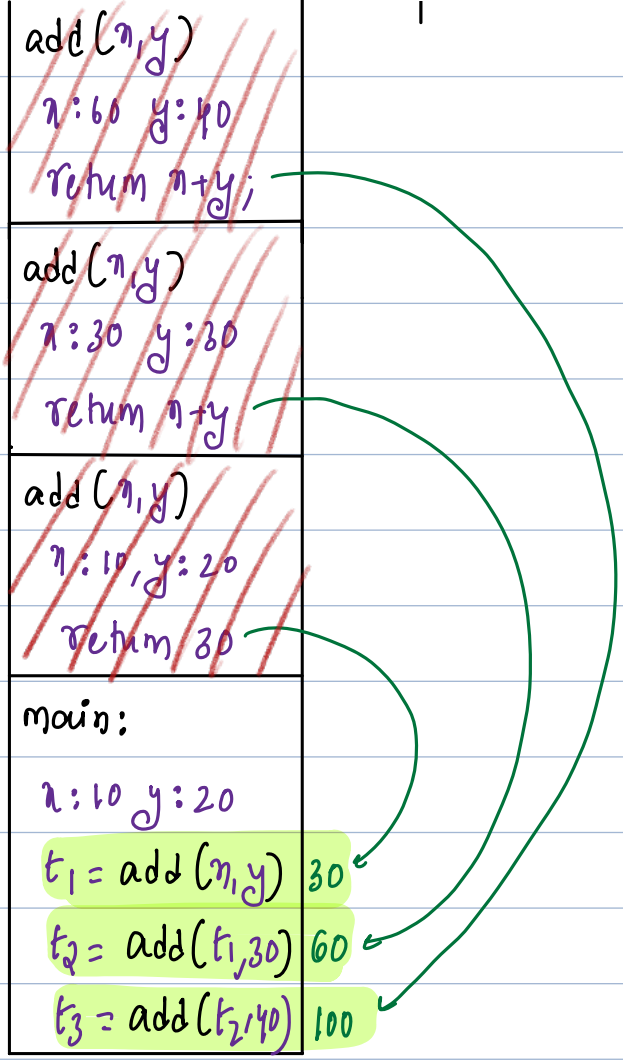


Note: Once function call is completed all memory allocated to it in stack will be erased.

Ex2:

```
static int add(int x, int y){
    return x+y;
}

public static void main(){
    int x=10, y=20;
    int t1 = add(x,y);
    int t2 = add(t1,30);
    int t3 = add(t2,40);
}   print(t1+t2+t3) // 190
```

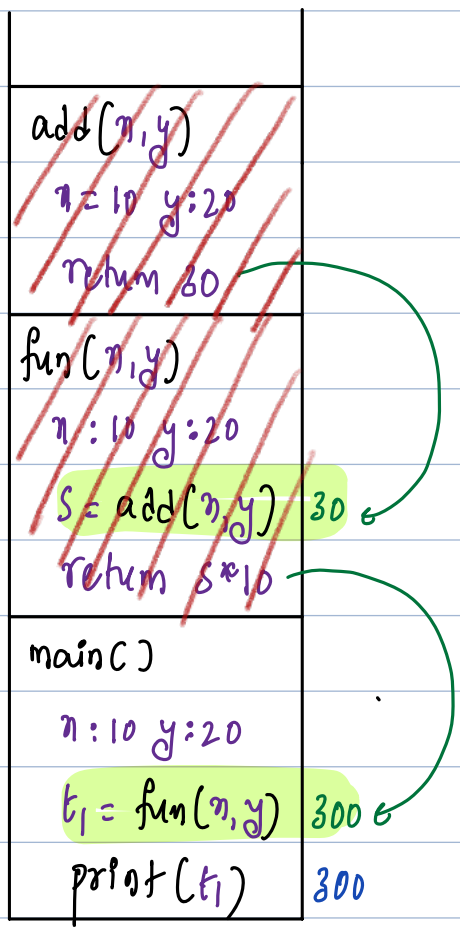


Ex3:

```
static int add(int x, int y){
    return x+y;
}

static int fun(int x, int y){
    int s = add(x,y);
    return s*10;
}

public static void main(){
    int x=10, y=20;
    int t1 = fun(x,y);
    print(t1);
}
```



Memory management in Java.

int/long/float/double ↗ variable which stores address:

Stack: All primitive datatypes & reference variable & Function calls stored in stack

Note: All variable rules applied to reference variable as well.

Heap: Container of that reference variable present in Heap

Eg: Array / ArrayList / Object stored in heap / ?

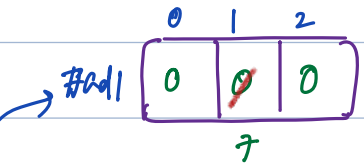
Eg:

```
public static void main(){
    int n=10;
    int[] ar = new int[3];
      ref var     creates arr
    println(ar) // ad1
    println(ar[2]) // 0
    ar[1]=7; //
}
```

stack

main:
n : 10
ar = ad1

heap



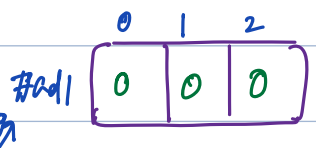
Eg2:

```
public static void main(){
    int n=10;
    int[] ar1= new int[3];
    int[] ar2 = ar1; //ad1
    println(ar1) //ad1
    println(ar2) //ad1
}
```

stack

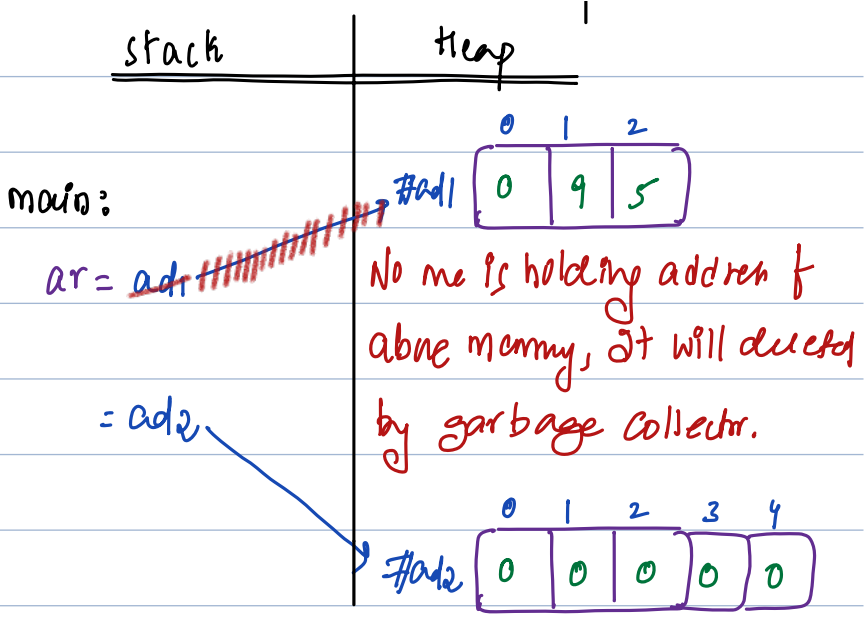
main:
n : 10
ar1 = ad1
ar2 = ad1

heap



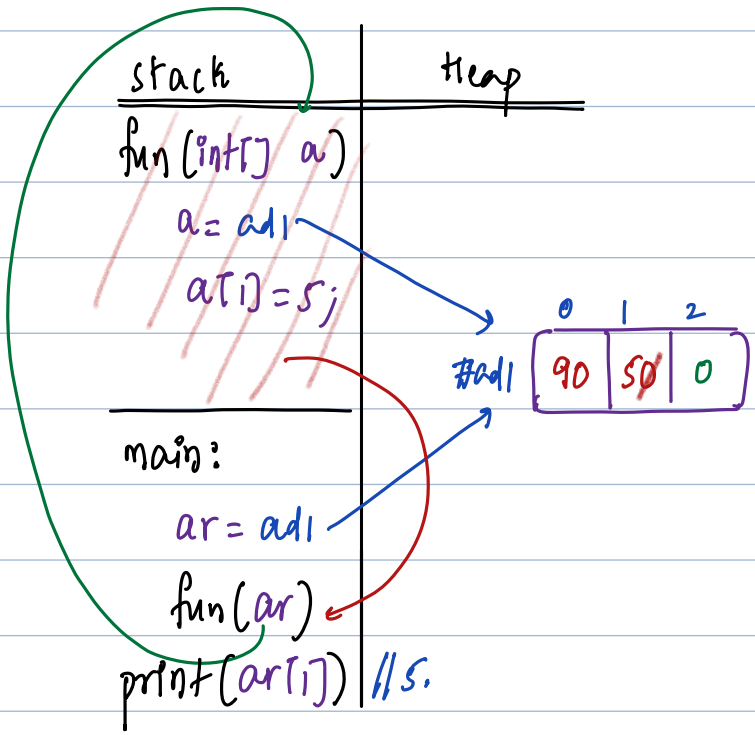
Eg3:

```
public static void main(){
    int[] ar = new int[3];
    println(ar) // add
    ar[1] = 9;
    ar[2] = 5;
    ar = new int[5]; ?
    println(ar)
}
```



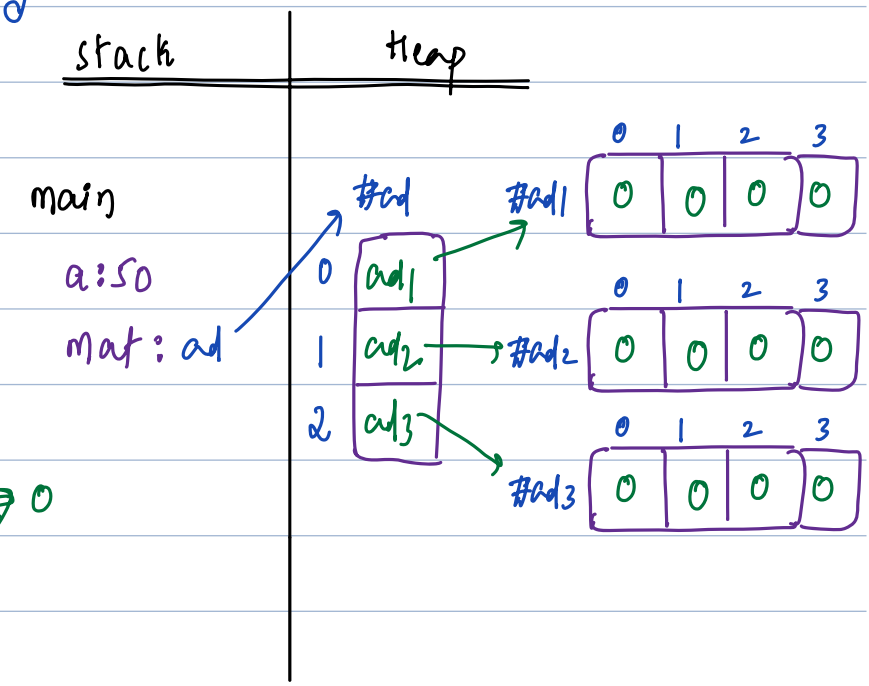
Eg4:

```
static void fun(int[] a){
    println(a)
    a[1] = 5;
}
public static void main(){
    int[] ar = new int[3];
    println(ar) // add
    ar[0] = 90;
    ar[1] = 50;
    fun(ar)
    println(ar[1])
}
```



Eg5: 2D Arrays : Combining 1D Arrays

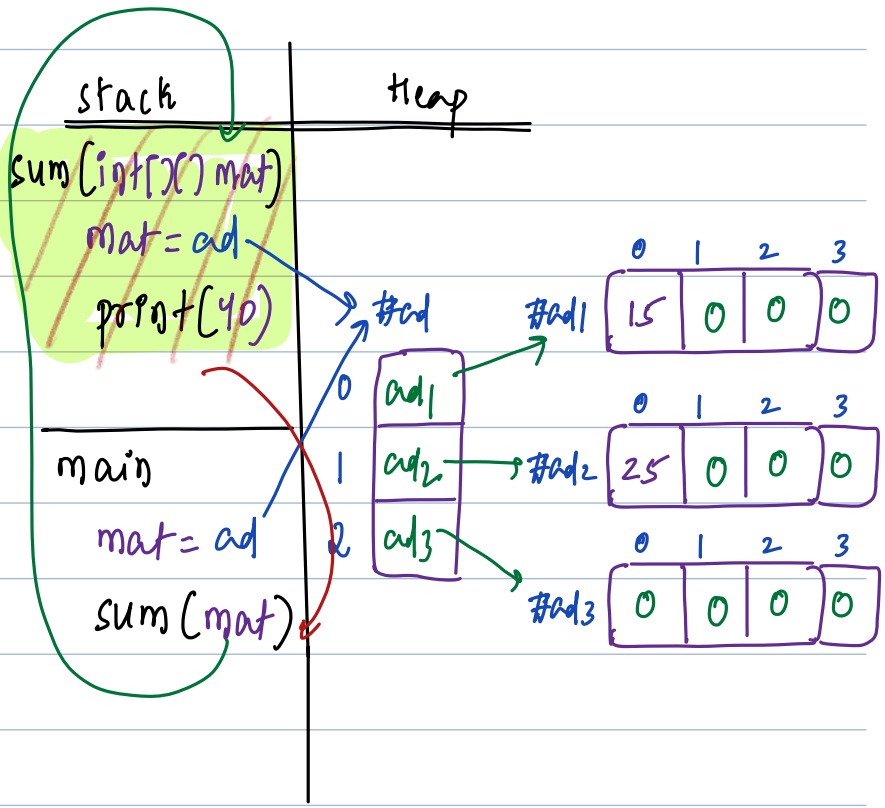
```
public static void main(){
    int a = 50;
    int[][] mat = new int[3][4]
    println(mat) // ad
    println(mat[1]) // ad2
    println(mat[1][3]) // ad2[3] ⇒ 0
}
```



Eg6:

```
static void sum(int[][] mat){
    println(mat);
    println( mat[0][0] + mat[1][0]);
}

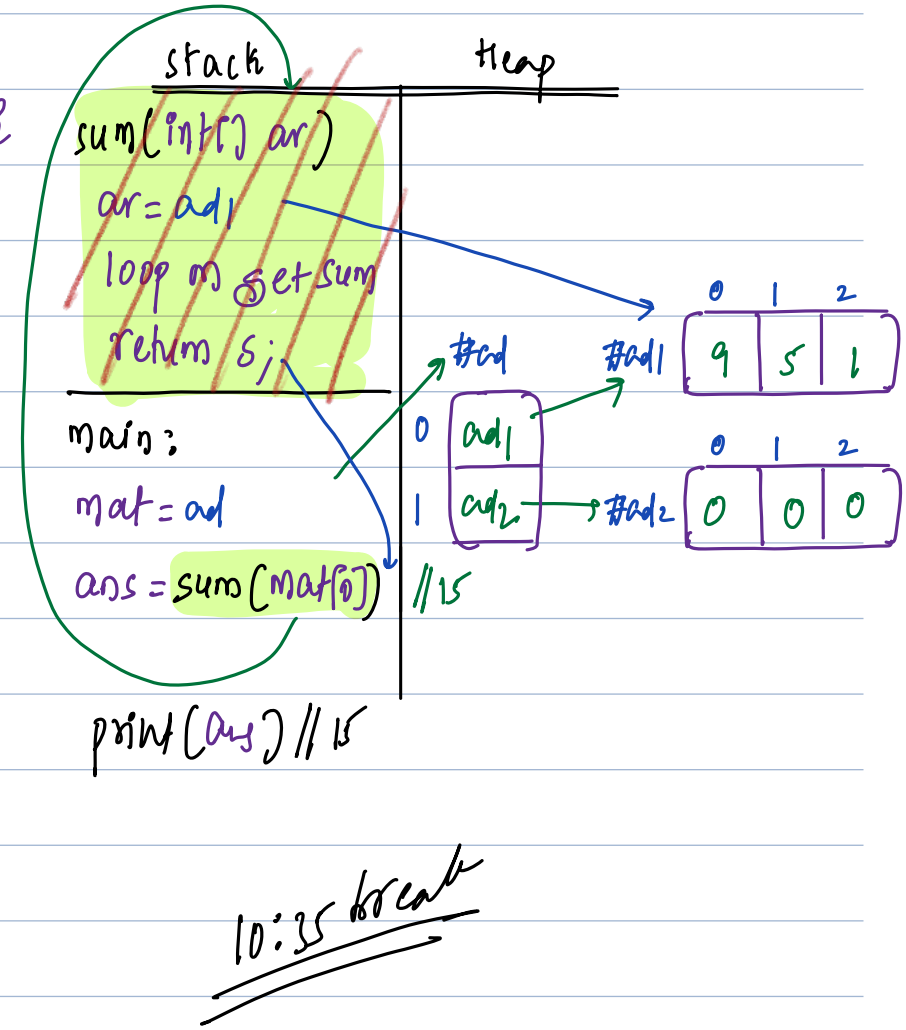
public static void main(){
    int[][] mat = new int[3][4]
    mat[0][0] = 15;
    mat[1][0] = 25;
    sum(mat)
}
```



Eg:

```
static int sum(int[] ar){
    println(ar)
    int s = 0;
    for(int i=0; i<ar.length; i++){
    }  s = s + ar[i]
    return s;
}

public static void main(){
    int[][] mat = new int[2][3] ✓
    mat[0][0] = 9; ✓
    mat[0][1] = 5;
    mat[0][2] = 1;
    int ans = sum(mat[0])
    println(ans)
}
```



10:35 break

Note:

```
1  public class MainClass {
2    public static void main(String[] args) {
3        int x = 10;✓
4        int[][] mat = new int[3][4];
5        System.out.println("Address of 2D"+mat);
6        int[] ar = new int[5];
7        System.out.println("Address of 1D"+ar);
8        int[]   =  int[][]
9        int[] cp = mat; // 2Dadrres --> Storing in 1Daddress
10       // 2D Address only in 2d reference varibale
11       // 1D Address only in 1d refrernce varibale
12
13
14
15   }
16 }
```

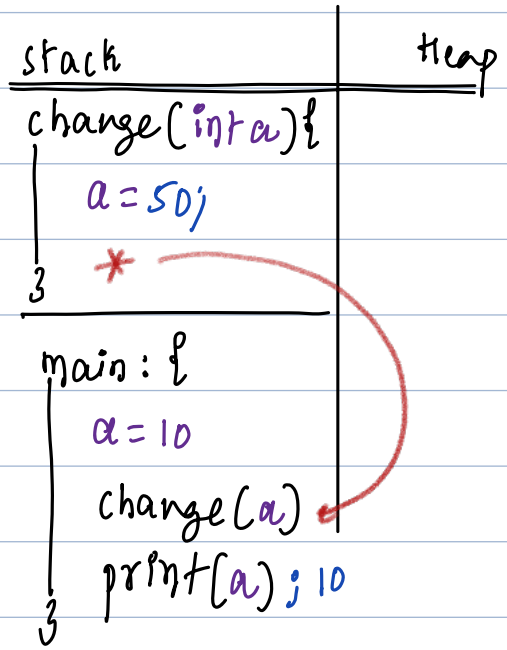
Error: incompatible types: int[][] cannot be converted to int[]

Q1:

Java

```
static void change(int a) {
    a = 50;
}

public static void main(String args[]) {
    int a = 10;
    change(a);
    System.out.println(a);
}
```

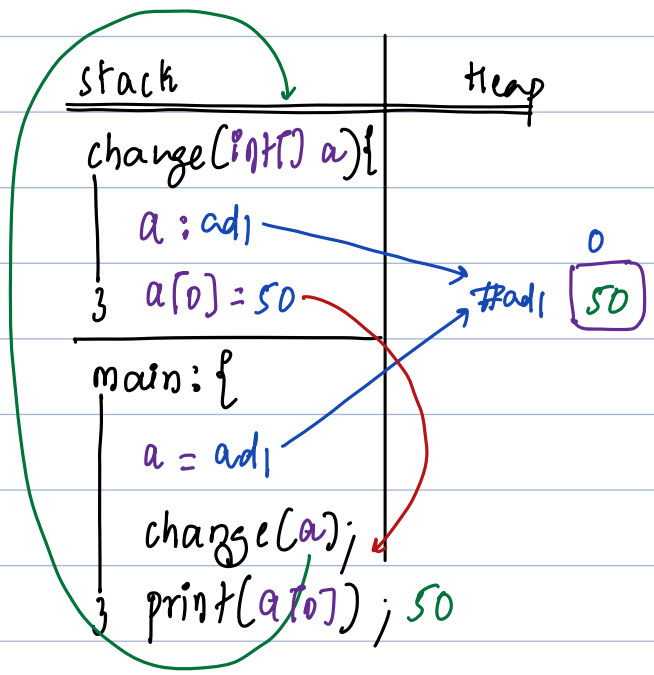


Q2:

Java

```
static void change(int[]a) {
    a[0] = 50;
}

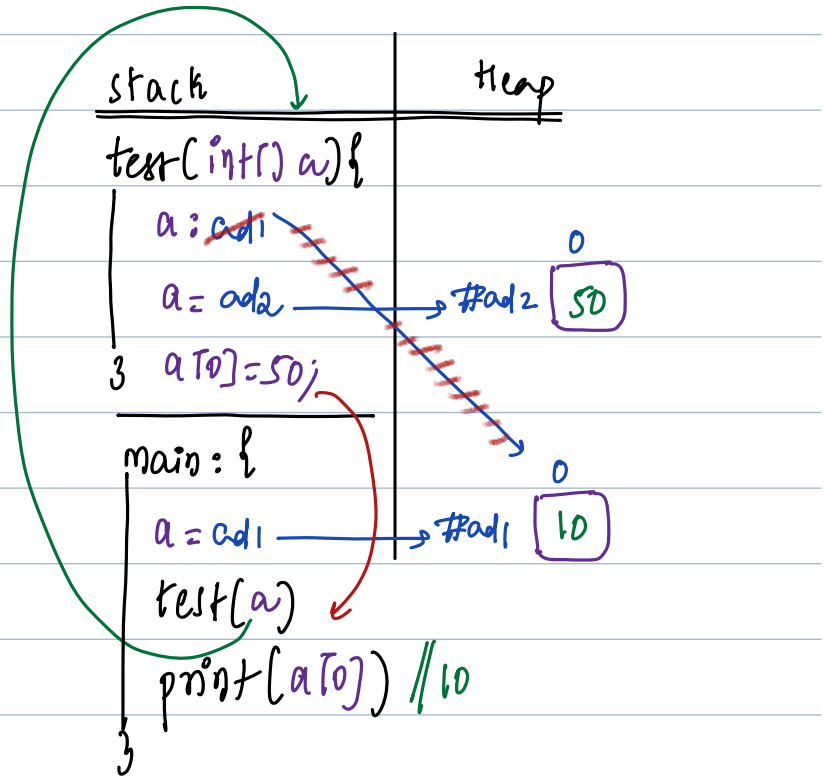
public static void main(String args[]) {
    int[]a = {10}; ✓
    change(a); ✓
    System.out.println(a[0]);
}
```



Q3:

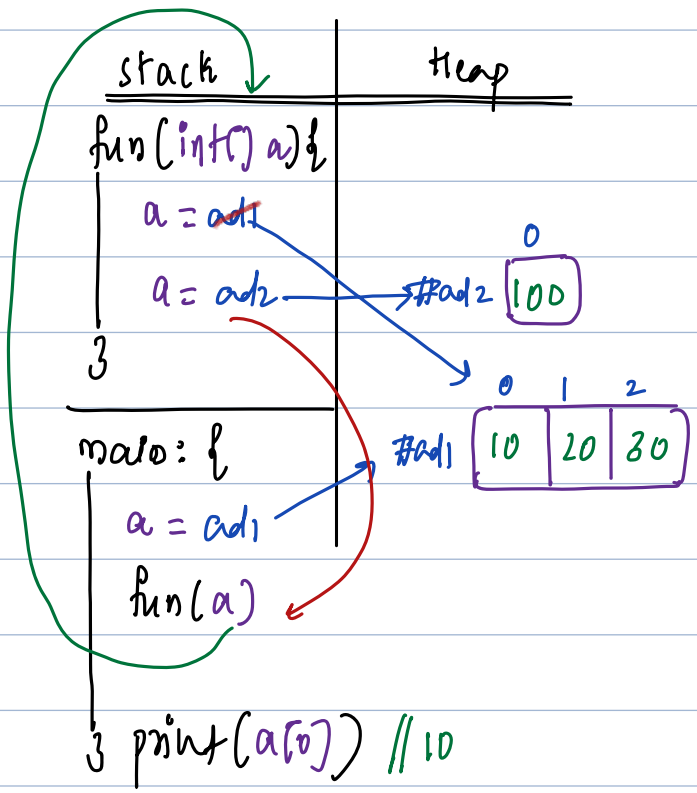
```java
static void test(int[]a) {
  a = new int[1];
    a[0] = 50;
}

public static void main(String args[]) {
  int[]a = {10};
  test(a);
    System.out.println(a[0]);
}
```



Q4:

```java
static void fun(int[] a) {
    a = new int[1];
    a[0] = 100;
}
public static void main() {
  int[] a = {10, 20, 30};
  fun(a);
    System.out.println(a[0]);
}
```



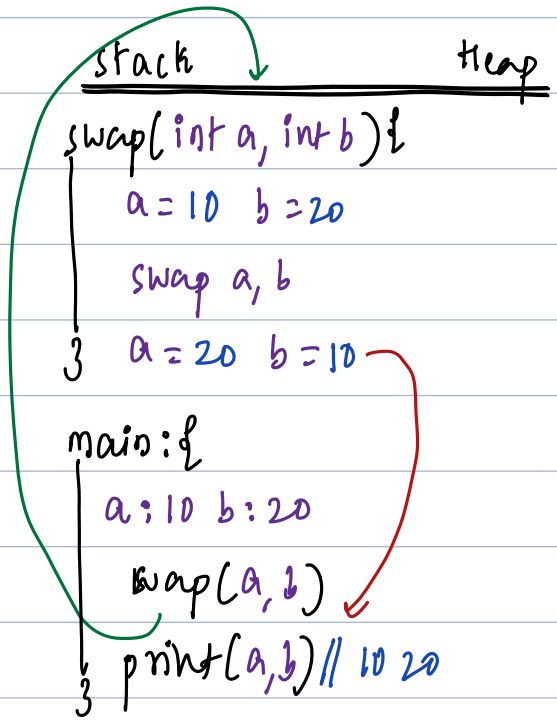
Q5:

```java
static void swap(int a,int b) {
    int temp = a;
    a = b;
    b = temp;
}

public static void main(String args[]) {
    int a = 10;
    int b = 20;
    swap(a,b);
    System.out.println(a + " " + b);
}
```

Different, based on scope, updating them in swap won't change in main.

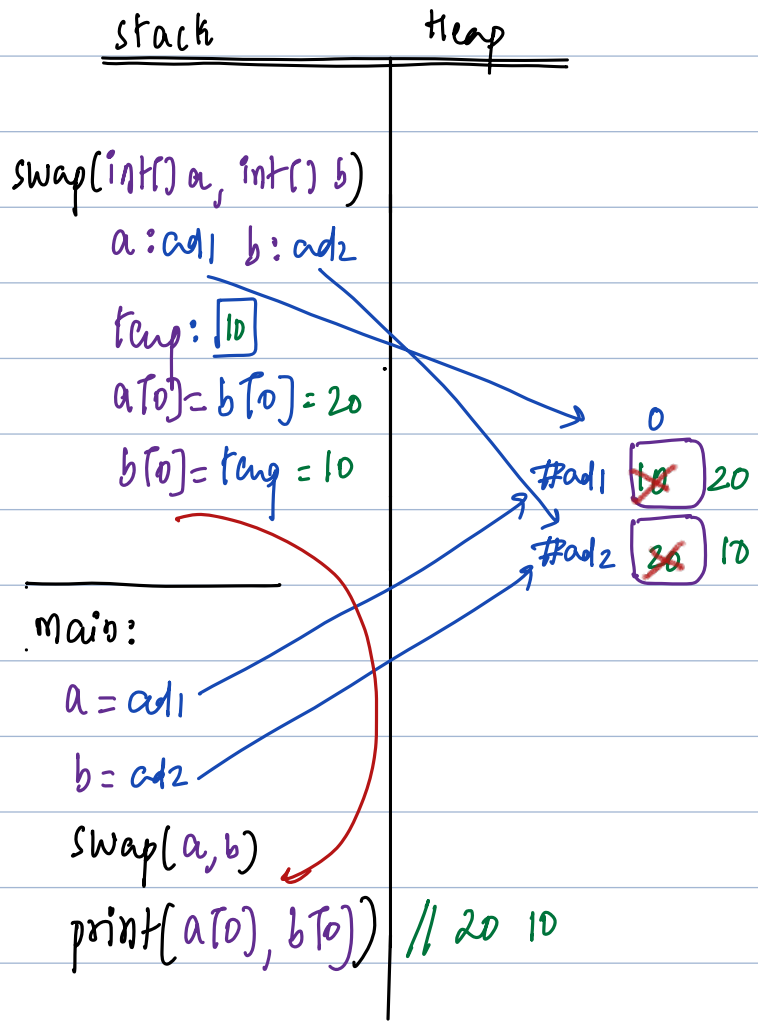
10 20



Q6:

```java
static void swap(int[]a,int[]b) {
    int temp = a[0];
    a[0] = b[0];
    b[0] = temp;
}

public static void main(String args[]) {
    int[]a = {10};
    int[]b = {20};
    swap(a,b);
    System.out.println(a[0] + " " + b[0]);
}
```



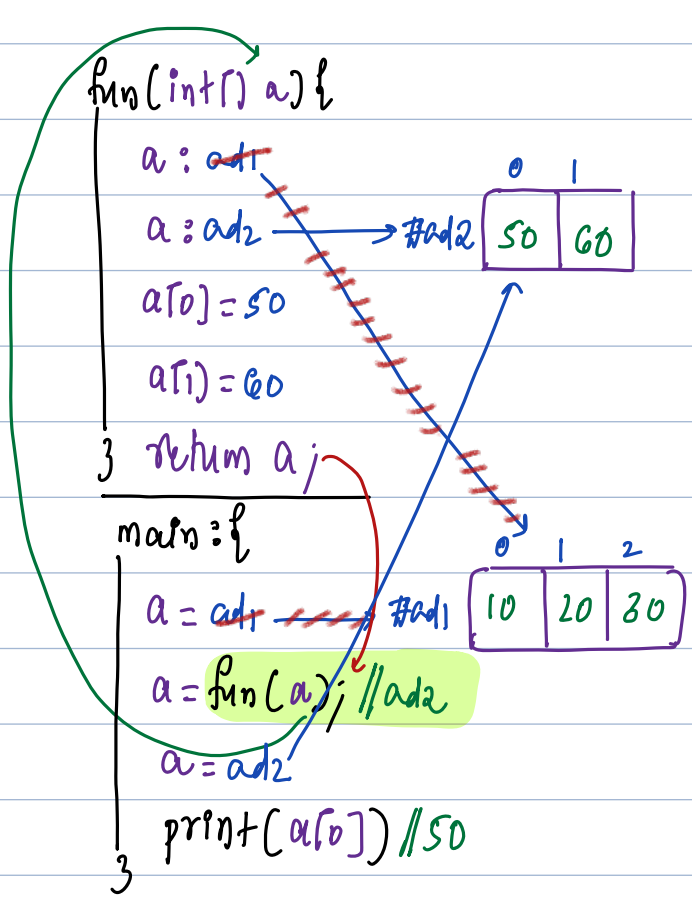
Q7.

Return 1D Reference variable.

Java

```java
static int[] fun(int[]a) {
    a = new int[2];
    a[0] = 50; a[1] = 60;
    return a;
}

public static void main(String args[]) {
    int[]a = {10,20,30};
    a = fun(a);
    System.out.println(a[0]);
}
```



Q: TODO

Java

```java
static void test(int[]a) {
    a = new int[2];
    a[0] = 94;
}

public static void main(String args[]) {
    int[]a = {10,20,30};
    test(a);
    System.out.println(a[0]);
}
```

Python

```python
def test(a):
    a = [0, 0]
    a[0] = 94

def main():
    a = [10, 20, 30]
    test(a)
    print(a[0])

if __name__ == "__main__":
    main()
```

stack heap

Difference Between .equals() and == in Java

Below is a table differentiating between .equals() and == in Java.

Aspect	==	.equals()
Purpose	Used to compare primitives or check if two references point to the same object.	Used to compare the content or state of two objects.
Usage	For primitive data types, it compares the values. For object references, it compares the reference locations (memory addresses).	Compares the values stored in two objects, based on the implementation of the .equals() method in the object's class.
Default Behavior	For objects, checks reference equality. For primitives, checks value equality.	In the Object class, it behaves the same as ==, checking reference equality. However, it is often overridden in child classes to check content equality.
Overridable	**No**, **==** is an operator and its behaviour cannot be changed.	**Yes**, Classes can override the **.equals()** method to define their own criteria for equality.
Primitive Comparison	Applicable and compares values directly.	Not applicable directly to primitives. Primitives must be wrapped as objects (e.g., Integer, Character) to use .equals().
Object Comparison	Checks if two reference **variables** point to the same object instance.	Checks if the contents of two objects are logically equivalent, depending on the implementation of .equals().

What is .equals() in Java?

The **.equals() method in Java** is used to compare two objects for equality based on the data or content of the objects rather than their reference identities. By default, the **.equals()** method in the Object class compares the memory addresses or references of the objects, essentially checking if the two references point to the same object in memory. However, the **.equals()** method is often overridden in user-defined classes to perform a content-based comparison, checking if the data within two objects is the same.